PHÂN TÍCH & TỐI ƯU XE DÒ LINE

ESP32 + 2 cảm biến TCRT5000

Tài liệu hướng dẫn nhanh cho sinh viên - tập trung vào bất cập, cách sửa và hướng phát triển

Mục tiêu Các em cần hiểu vì sao xe đang lắc, vì sao cua góc chưa ổn, và nên sửa theo thứ tự nào. Chưa cần lao ngay vào PID; trước tiên phải làm cho cảm biến, motor và logic điều khiển cơ bản ổn định.

Repo tham khảo: github.com/Sune27/line_follower_robot_arm

1. Vấn đề lớn nhất nằm ngay ở tốc độ hai bánh

Code hiện tại giảm bánh trong quá mạnh và tăng bánh ngoài khá nhiều. Ví dụ với tốc độ cơ sở 45%, khi cần chỉnh hướng có thể thành khoảng 11% - 51%. Với motor TT vàng, 4-11% PWM rất dễ rơi vào vùng không đủ mô-men để bánh quay ổn định.

- Hậu quả: lúc bánh quay, lúc không quay; xe bẻ quá tay rồi sửa ngược lại -> lắc liên tục.
- Cua góc sẽ "hên xui" vì vùng PWM thấp phụ thuộc pin, tải, ma sát và từng motor.

Việc đầu tiên nên làm Giảm tốc độ cơ sở và giảm chênh lệch hai bánh. Có thể bắt đầu ở khoảng: đi thẳng 35-35; chỉnh trái 25-42; chỉnh phải 42-25. Sau đó mới tinh chỉnh theo xe thật.

2. Code hiện tại chưa phải PID

Phần dò line đang đọc trạng thái số 0/1 từ hai cảm biến và xử lý theo các trường hợp 00 / 10 / 01 / 11. Đây là điều khiển theo luật (rule-based/bang-bang), chưa có error, P, I, D thực sự.

- Bài nhập môn dùng rule-based là hoàn toàn được, miễn là mô tả đúng trong báo cáo.
- Nếu muốn xe chạy mượt hơn, nên chuyển dần sang tín hiệu analog và PD.

3. Hai chân AO của TCRT5000 đang bị bỏ phí

Nếu chỉ dùng DO, ESP32 chỉ biết trắng/đen. Trong khi AO cho biết mức phản xạ liên tục, tức là có thể biết cảm biến đang ở xa line, sát mép line hay nằm trên vùng đen.

```
Gợi ý:  AO Left -> GPIO34
         AO Right -> GPIO35
         VCC cảm biến -> 3.3 V (hoặc phải đảm bảo AO không vượt mức an toàn của ESP32)
```

Khi có giá trị analog, ta mới tạo được error liên tục và "lệch ít sửa ít, lệch nhiều sửa nhiều". Đây là bước chuyển quan trọng để xe bám line mượt hơn.

4. Dùng PD trước, chưa cần PID

Với xe dò line nhập môn, thành phần P giúp kéo xe về line, còn D giúp giảm hiện tượng sửa quá tay. Thành phần I thường chưa cần ở giai đoạn này.

```
error = left - right
correction = Kp*error + Kd*(error - previous_error)
left_speed  = base - correction
right_speed = base + correction
```

- Tăng Kp từ nhỏ đến khi xe bám line nhưng bắt đầu hơi dao động.
- Sau đó tăng Kd từ từ để giảm lắc.
- Không copy Kp/Kd trên mạng rồi dùng nguyên xi; phải tune theo xe thật.

5. Hai cảm biến có giới hạn khi gặp cua 90°

Đây là hạn chế về thông tin cảm biến, không phải chỉ là lỗi code. Với hai mắt đặt hai bên line, trạng thái "cả hai đều trắng" có thể có hai nghĩa hoàn toàn khác nhau:

```
TH1: xe đang đúng giữa line  -> cả hai mắt đều trắng
TH2: xe đã mất line          -> cả hai mắt vẫn có thể trắng
```

Do đó khi cua gắt, xe có thể chạy quá điểm cua và chương trình vẫn tưởng đang đi đúng. Không có PID nào khắc phục hoàn toàn việc thiếu thông tin này.

6. Cần giữ "trí nhớ hướng cua" tốt hơn

Code đã có ý tưởng lưu hướng vừa nhìn thấy line (LEFT/RIGHT), nhưng không nên reset quá sớm về STRAIGHT ngay khi thấy 00. Nếu đang trong pha cua, 00 có thể chỉ là mất line tạm thời.

> **Nguyên tắc** Chỉ cập nhật last_turn khi thật sự thấy 10 hoặc 01. Khi mất line ngắn, tiếp tục tìm theo hướng vừa cua trong một khoảng thời gian nhỏ thay vì lập tức đi thẳng.

7. Nếu vẫn giữ 2 mắt DO: sửa logic trước

```
BASE = 35
TURN_INNER = 25
TURN_OUTER = 42

00 -> đi thẳng
10 -> chỉnh trái nhẹ, last_turn = LEFT
01 -> chỉnh phải nhẹ, last_turn = RIGHT
11 -> cua mạnh theo last_turn hoặc xử lý vạch đặc biệt
```

Quan trọng: đừng để bánh trong tụt xuống vùng PWM quá thấp nếu motor không còn quay ổn định.

8. Thêm lọc cảm biến

Ở mép line, DO có thể nhảy 0-1 rất nhanh. Nếu mỗi lần nhảy là đổi tốc độ motor ngay thì xe sẽ rung.

```
Đọc 5 mẫu liên tiếp -> lấy giá trị chiếm đa số
Ví dụ: 0 1 1 1 0  => coi là 1
```

- Có thể đọc cách nhau khoảng 1 ms; chỉ cần bộ lọc đơn giản để giảm chattering.

9. Tách nhịp điều khiển khỏi Wi-Fi/telemetry

Vòng điều khiển line nên chạy đều và nhanh hơn các tác vụ hiển thị/truyền dữ liệu. Nếu cùng một vòng lặp còn xử lý Wi-Fi, TCP và dashboard thì chu kỳ điều khiển sẽ không ổn định.

```
Line control : 10-20 ms
Telemetry    : 100-200 ms
Wi-Fi status  : 1000 ms hoặc chậm hơn
```

10. Kiểm tra lại TB6612FNG

Nếu dùng TB6612FNG thì nên hiểu đúng chức năng chân: AIN1/AIN2 và BIN1/BIN2 quyết định chiều; PWMA/PWMB quyết định tốc độ; STBY phải ở mức bật. Nếu PWMA/PWMB hoặc STBY bị để floating thì xe sẽ chạy không ổn định.

Khuyến nghị Dùng đúng cấu trúc driver: direction bằng IN1/IN2, tốc độ bằng PWM ở PWMA/PWMB. Cách này vừa chuẩn vừa dễ giải thích cho sinh viên.

11. Hạn chế dùng GPIO12 cho motor nếu còn chân khác

GPIO12 là chân có vai trò strapping trên ESP32. Nó không phải nguyên nhân làm xe lắc, nhưng có thể gây các lỗi boot khó hiểu nếu bị kéo mức không phù hợp. Với project sinh viên, nên ưu tiên các GPIO thông thường còn trống.

12. Nếu được thêm 1 cảm biến: dùng 3 mắt

Đây là nâng cấp hiệu quả nhất nếu mục tiêu là chạy cua chắc hơn nhưng vẫn giữ bài toán đơn giản.

```
Left   Center   Right
  •       •       •

010 -> đi thẳng
100/110 -> lệch trái
001/011 -> lệch phải
000 -> mất line
111 -> giao nhau / vạch đặc biệt
```

Với 3 mắt, trạng thái “đang giữa line” và “mất line” đã tách biệt. Khi đó dùng rule-based hoặc PD đều dễ hơn và ổn định hơn nhiều.

Phương án mình chọn cho đúng project này

Các em không cần sửa tất cả cùng lúc. Hãy làm theo thứ tự dưới đây để biết chính xác thay đổi nào giúp xe tốt hơn.

Bước	Việc làm	Mục đích
1	Giảm base xuống khoảng 30-35%	Ổn định xe trước
2	Giảm độ chênh tốc khi cua	Hạn chế overshoot, giảm lắc

Bước	Việc làm	Mục đích
3	Đo PWM tối thiểu từng motor bắt đầu quay	Tránh vùng chết motor
4	Lọc DO 3-5 mẫu	Giảm nhiễu ở mép line
5	Giữ last_turn khi mất line ngắn	Cải thiện cua gắt
6	Tách nhịp line khỏi telemetry/Wi-Fi	Chu kỳ điều khiển đều hơn
7	Chuyển AO -> ADC và làm PD	Bám line mượt hơn
8	Nếu cần cua 90° chắc chắn: thêm mắt giữa	Phân biệt đúng line và mất line

Kết luận cho buổi lab Nếu chỉ có một buổi để sửa xe, ưu tiên bước 1 -> 5 trước. Khi xe đã chạy ổn bằng DO rồi mới nâng lên AO + PD. Nếu bài thi có nhiều cua 90°, thêm cảm biến giữa sẽ hiệu quả hơn việc cố viết PID phức tạp với 2 mắt.